Chapter I. Introduction

"The ultimate goals (somewhat utopian) include error-free compilers, ... ".

S. Greibach

1. Overview

In this thesis we describe the design, implementation, and verification of a compiler for a Pascal-like language. While previous work on compiler verification has focused largely on proofs of abstract "code generating algorithms" we are interested in a "real" compiler translating a string of characters into machine code efficiently. We consider all components of such a compiler including scanning, parsing, type checking and code generation.

Our interest is twofold. First, we are concerned with the formal mathematical treatment of programming languages and compilers and the development of formal definitions suitable as input for automated program verification systems. Second, we are interested in the more general problem of verifying large software systems.

There are several reasons for verifying a compiler. Compilers are among the most frequently used programs. Consequently, if we invest in program proofs it is reasonable to do this in an area where we may expect the highest payoff. Verification techniques are in general applied to programs written in high level languages. If we ever want to use verified programs we have to be able to correctly translate them into executable machine languages; another reason why correct compilers are a necessity.

The implemented language, *LS*, contains all features of Pascal [JW76] that are of interest to compiler constructors. The language contains most control structures of Pascal, recursive procedures and functions, and jumps. It provides user defined data types including arrays, records, and pointers. A simple facility for input – output is included; each program operates with one input and one output file.

The target language, *LT*, assumes a stack machine including a display mechanism [RR64, Or73] to handle procedure and function calls. This language simplifies our task somewhat as it avoids the issue of register allocation and similar irrelevant

details. But at the same time the target language is realistic in that similar machine architectures exist, notably the B6700.

The compiler itself is written in Pascal Plus, a dialect of Pascal accepted by the Stanford verifier.[1] The Stanford verifier [SV79] is used to give a complete formal machine checked verification of the compiler.

We review existing methods for the formal definition of programming languages. We use the most appropriate definitional methods to formally define source and target language.

We further investigate how the correctness of a compiler can be specified in a form suitable for mechanical verification. Here, we have to deal with several technical issues such as fixed points and reasoning about pointers.

We show that an efficient program can be developed systematically from such specifications.

During this research verification has proven to be a most useful tool to support program development; verification should be an integral part of the development process and plays a role comparable to that of type ckecking in strongly typed languages. This methodology of verification supported programming is not limited to compilers, rather it is applicable to arbitrary problem domains.

The results of this thesis are encouraging and let us hope that verification will soon become a widely accepted software engineering tool. But also this work reveals many of the trouble spots still existing in today's verification technology. We point to several promising research areas that will make verification more accessible. The need for better human engineering and integrated software development systems is particularly urgent.

2. Program verification

Verification has provoked several controversial statements by opponents and proponents recently [DL79]. Therefore it is appropriate at this point to clarify what verification is, what it can do, and, most importantly, what it cannot do.

To verify a program means to prove that the program is consistent with its specifications. Subsequently, the term "verification" is used as a technical term referring to the process of proving consistency with specifications. A program is "verified" if a consistency proof has been established.

Formal specifications for a program can express different requirements. For example, a specification can be "the program terminates for each set of input data".

1.) Notable difference to standard Pascal is that formal documentation is an integral part of the language, for more details see 2.3.

Or, even more trivially, one can specify that "the program satisfies all type and declaration constraints"; every compiler verifies this property. But of course, we consider more interesting properties of our compiler. What exactly its specifications are is discussed in the following sections and in more detail in chapter IV.

Since specifications can be weak and need not (and generally do not) express all requirements for a program verification should not be confused with correctness in the intuitive sense (i.e. the program does what one expects it to do). Verification is not a substitute for other software engineering techniques such as systematic program development, testing, walk-through and so on; rather verification augments these techniques. It gives us an additional level of confidence that our program has the property expressed in its specifications.

Depending on the application of a program certain errors may be mere annoyances while other may have disasterous effects. We can classify errors as "cheap" and "expensive". For example, in terms of our compiler cheap properties are the reliablility of error recovery and termination. An expensive error would be if the compiler would translate an input program without reporting an error but would generate wrong code.

As long as verification is expensive we can concentrate our efforts on the most costly requirements of a program. These requirements can be formalized and taken as specifications for the program. Verification can be used to guarantee these expensive requirements. Other cheaper properties can be validated by conventional testing methods. Redundant code can easily be added to a verified program to increase its reliability or establish additional properties without effecting verified parts of the program. For example, in our compiler a sophisticated error recovery could be added without invalidating program proofs.

2.1. Writing correct programs

The non-technical use of the expression "to verify a program" suggests that there is an existing program which we subject to a verification. This is possible in principle but most certainly not practical for large software systems; it will only work for "toy" programs.

Instead, verification should be an integral part of program development: a program and its proof should be developed simultaneously from their specifications.

2.1.1. Program design

The process of designing a verified program can be visualized by figure 1.
The starting point for the program design is a formal specification of the problem. In general, this specification alone is insufficient to prove the correctness of an

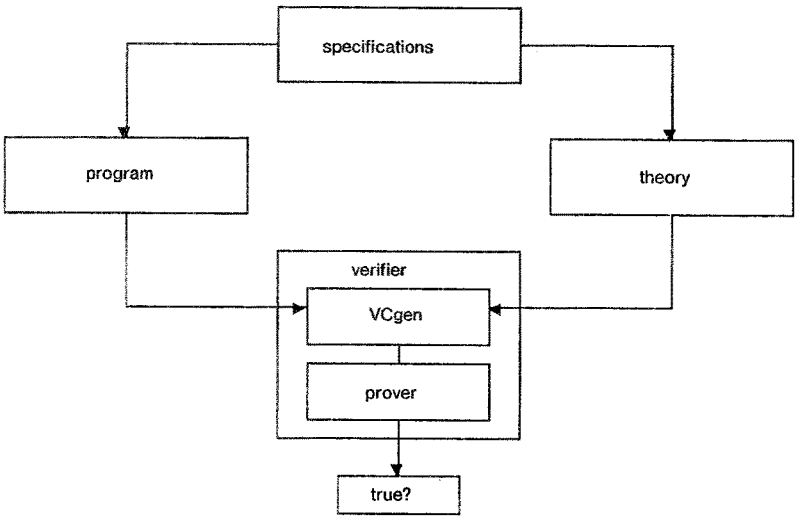


Fig. 1

implementation. We need additional theorems and lemmas about properties of the problem domain which enable an efficient implementation. We refer to these theorems and lemmas together with the formal specifications as "theory" or logical basis for our program proof.

The original specifications as well as derived lemmas and theorems are initially given in some suitable mathematical language. They have to be encoded in a form accepted by the verification system used. This step may not be as trivial as it appears on first sight. Problems arise if the "assertion language" used by the verification system is less expressive than the mathematical language. In chapter II we deal with the problem of encoding the language definition in our assertion language; of particular interest is the treatment of the least fixed point operator.

Given a suitable theory we implement a program in the style of structured programming [Di76]. Starting with a rough outline of the program we successively refine it until we derive an executable version. We start out with a first draft of our program where we are merely interested in the overall structure; all implementation details are omitted. The initial draft is then checked for consistency by the verification system. If consistency is not given, then either the program has to be corrected or stronger lemmas have to be added to the logical basis. After a successful verification we are sure that our initial design is correct (with respect to its specifications). Observe that this situation differs crucially from that in ordinary

top down programming. In the latter case we have no means of validating an initial design until the refinement of the program is completed down to the lowest level and the program can actually be put on a machine and tested.

Given a verified first draft, we refine this program by implementing some of the points still left open. This step may require additional theorems and lemmas to be proven to enable efficient implementation and follows along the same lines as the initial design. We repeat this process until we arrive at an executable program.

2.1.2. An example

Let us explain the importance of the theory with a trivial *gcd* example. Our specification might be to implement a function $f(x, y)$ such that $f(x, y) = gcd(x, y)$. If we are given the following formal definition of *gcd* an efficient implementation of f is not easily derived:

$$gcd(x, y) = max\{z \mid x \bmod z = 0 \wedge y \bmod z = 0\}.$$

The straightforward implementation suggested by this definition is to compute the set given in the definition and then search for the maximum. A much more efficient program is possible if we can prove additional theorems; for example $gcd(x, y) = gcd(x \bmod y, y)$.

In general, any efficient program is based on a mathematical property of the problem domain. The really creative part of programming is to uncover relevant properties of the specifications that lead to good implementations.

In many cases we can resort to well known results in literature. But note, that if we prove a theorem, then this is a purely mathematical activity and has nothing to do with mechanical program verification. For example, in the case of the *gcd* above relevant theorems are part of number theory. In the future it is to hope that we have proof checking systems which can aid us in this part of program development.

People unfamiliar with verification often do not realize the importance of theory in writing programs. Programming is misleading in this respect because it allows us to write programs without having completely understood the problem. Verification makes us aware of all the mathematical properties (or alleged properties) we use in a program. A frequent complaint about verifications is that even simple problems require a discouraging amount of mathematics. We feel that this is inherent to programming: if we want correct programs we have to understand the problem and ultimately only a mathematical treatment can help us in this.

Although "toy" verification examples are often mathematically very subtle;[2] many programs written in practice are based on much simpler foundations.

2.) For example dealing with permutations [Po79]

2.1.3. A word of warning

A verified program may fail to execute safely for several reasons. Firstly, a program proof could be wrong. There are two main causes for this. It is most likely, that an error occured in manually proving theorems and lemmas that are the basis of the implementation. It can be hoped, that in the future these steps are amenable to automation or can at least be checked with proof checking systems. But then of course, there is the verification system itself which could be wrong. People have pointed out, that trying to verify the verifier is a vicious circle and cannot work. True, but, we have a much simpler way out. We can take our program with formal specifications and subject it to a different verification system. Unless both verifiers contain the same bug this will reveal any error.

Secondly, even if the program proof were valid the program will ultimately be executed in a real world environment with all its uncertainties.

Still, verification is an extremely promising tool to fight today's "software crisis". It is a tool to increase our confidence in a program to an extent not achievable with any other known means.

2.2. Logics of programming

So far we have discussed how to use verification during program development. We now look into the details of automatic verification systems. First, we give a general overview over existing formal systems that allow mechanical verification and discuss their advantages and disadvantages. We then take a closer look at the Stanford verifier and describe its use.

The focus of our attention is on three methods for verifying programs. These methods are selected because they appear to be reasonably well developed and more importantly, there are implementations of verification systems based on these methods.

2.2.1. Logic of computable functions (LCF)

LCF [Mi72a] is a formal system based on Scott's logic of computable functions.[3] Since in LCF programs are objects of the formal language it is possible to express very general theorems about programs in LCF. Examples are program equivalence, program transformations as well as consistency and termination.

Although LCF is best suited to reason about denotational language definitions [AA74, AA77, He75] it is also capable of expressing Hoare style proof rules (see below).

3.) Details of Scott's logic are discussed in chapter II.

Semi-automatic proof checking systems for LCF have been implemented [Mi72b, GM75] and used for several program proofs [MW72, Co79a, Co79b, Ne73].

2.2.2. First order logic

In [Ca76, CM79] McCarthy and Cartwright propose to consider recursive programs as objects in a first order language. The logical system has to be augmented with an undefined symbol to account for nonterminating computations; recursive function definitions are translated into equations in the logic. Suitable induction schemas allow reasoning about least fixed points. The system has the drawback that it is restricted to first order functions. Furthermore punning between logic and programming language severely restricts the languages that can be handled and at this point the method seems not suitable to prove properties of Algol–like programs. FOL [WT74] is a proof checking program that allows mechanized reasoning about first order logic. FOL has been used for several program proofs based on McCarthy's and Cartwright's method.

2.2.3. Hoare's logic

In [Ho69] Hoare devised a deductive system[4] in which programs together with specifications can be derived. The verification problem is to determine if there exists a derivation of the given program together with its specifications. Given certain intermediate documentation (invariants) there is an effective way to construct a set of first order formulas which entail that such a derivation exists. Thus, if all these formulas (verification conditions) can be proven, the program is consistent with its specifications.

Hoare's logic is limited to partial correctness proofs; termination can be expressed by modifying the input program to include counters for all loops and suitable assertions as is outlined in [LS75, MP73]. Here, too, is a pun in that one considers expressions in the programming language to be terms in the underlying logic. This effectively excludes certain language constructs, e.g. expressions that cause side effects. The method requires the programmer not only to give input and output specifications but also specifications at intermediate points in the program.

2.3. The Stanford verifier

All of the methods mentioned above are interrelated (see for example [HL74, Do76, Po80]); however, most existing verification systems are based on Hoare's logic. This logic is used exclusively in this thesis.

4.) building on [Fl67]

The Stanford verifier [SV79] was used to mechanize the compiler proof. Parts of the proof are specifically tailored towards this system. We will point out those parts and it should be obvious how to adapt program structure, assertions, and proofs to other verification systems.

This section gives a brief overview over the Stanford verifier, subsequently referred to as "the verifier". For more details the reader should consult [SV79]. The verifier accepts standard Pascal with minor syntactic changes. In addition certain specifications are required, some are optional. The verifier uses the (formally) documented program to generate a set of first order formulas (verification conditions) based on a set of Hoare style proof rules [HW73]. If all verification conditions are true, the program is consistent with its specifications.

Note, that we distinguish "proving verification conditions" from "proving theorems and lemmas". The former is done completely mechanically by the verifier's prover while the latter refers to manual proofs. Occasionally we talk about "meta theorems". A meta theorem is a theorem about Hoare's logic itself. Examples are weak $\forall$–introduction and special proof rules about certain pointer operations to be introduced in later chapters.

2.3.1. Assertion language

In the language accepted by the verifier the following documentation exists:

- *entry* and *exit* assertions are required for each procedure and the main program describing input and output predicates.
- An *invariant* is required for each loop.
- All loops constructed with jumps have to be cut by at least one *assert* statement, however, additional *assert* statements may appear at arbitrary places in the program.
- A *comment* statement is similar to an *assert* statement except that it does not break the path through the program where it is attached, rather it requires additional specification to be proven (see [SV79]).

The assertion language is quantifier free first order logic with arbitrary function and predicate symbols. All free variables in assertions have to be variables of the program. Special terms are added to express operations on complex data structures,

- $< A, [i], e >$ where A is an array denotes A with $A[i]$ replaced by e.
- $< R, .f, e >$ where R is a record with field f denotes R with $R.f$ replaced by e.

A similar notation is used for operations on pointers [LS76]; this is discussed in chapter II in greater length.

An important concept is the use of "virtual" code. The use of "auxiliary" variables in program verification has been pointed out in earlier work [LS75]. The concept of virtual code is much more general however. A virtual variable is one whose value at no point influences the value of "real" variables. We put no restrictions on the use of virtual variables. They may appear at any point where real variables appear, subject only to the restriction that they don't influence the outcome of a real computation. Furthermore we allow for virtual procedures, functions, types etc. It is shown in chapter IV how virtual code is useful in expressing the correctness of an algorithm in a natural way.

2.3.2. Theorem Prover

The verifier's built-in theorem prover contains complete decision procedures for various theories and their combination [NO78]. In addition it allows the user to specify "rules" axiomatizing free function and predicate symbols. Each rule expresses some logical axiom. Through different syntactic formats of rules heuristic information is conveyed to the prover how to use this particular fact in a proof. The logical language used for rules is essentially the same as for assertions within the program except that free variables are considered universally quantified.

The set of axioms and lemmas expressed by the rules required for a program proof is called the *logical basis* of this proof. Proofs are relative to their logical basis.

There are three different rule formats accepted by the verifer's theorem prover. Each rule expresses a certain logical fact. In addition different rule formats imply certain heuristics to the theorem prover. We briefly describe, the logical contents of the different rule formats; we are not concerned with the heuristic aspect of rules.

A *replace* rule[5] of the form

$$\textbf{replace } T_1 \textbf{ where } F \textbf{ by } T_2$$

is equivalent with $F \Rightarrow T_1 = T_2$. The **where** clause may be missing in which case $F \equiv \mathit{true}$ is assumed. A rule of the form

$$\textbf{infer } F_1 \textbf{ from } F_2$$

means $F_2 \Rightarrow F_1$. Similarly, the rule

$$\textbf{from } F_1 \textbf{ infer } F_2$$

means $F_1 \Rightarrow F_2$. The last two rule formats may appear with a *whenever* clause. Whenever clauses contain heuristic information for the theorem prover; they do not change the meaning of a rule.

5.) The name replace rule is misleading since the theorem prover does not implement a "rewrite system".

3. Formal definition of programming languages

In order to be able to specify a compiler we need a method to formally define programming languages. Although there exists extensive literature on how to formalize language definitions this work seems not generally accepted; in most language designs informal english definitions prevail. The only exceptions are the by now generally accepted context free grammars. Many of the errors in current software can be attributed to a lack of specification. In the case of programming languages insufficient definitions leave ample room for major confusion of the user as well as the implementor. One result is the "try it out" attitude of many programmers who use the compiler rather than the manual as language definition. Also much of what is perceived as a compiler error is simply due to insufficient or ambiguous specification of languages.

In this section we briefly review various existing formal methods for the specification of programming languages. This section is far from being complete; rather the main emphasis is to motivate the particular choice of definitional formalisms used in the remainder of this thesis.

We adopt the common division of a language into syntax and semantics. The syntax describes how a program is externally represented as a string of characters while the semantics defines what a program means. Syntax is further subdivided into "micro syntax" or "token syntax" and the phrase structure of programs. Semantics is divided into "static" and "dynamic" semantics. Roughly, the former describes when a program is semantically valid; that is, it formalizes all type and declaration constraints. The dynamic semantics describes what the meaning of a semantically valid program is. Exactly what we mean by the meaning of a program depends very much on the particular formalism; we elaborate on this later. In addition to the components of a language definition outlined so far we introduce "tree transformations" which describe the relation between the phrase structure of a program and an "abstract" program which is input to the semantic analysis phase. This step may or may not be explicit in a particular formalism.

3.1. Syntax

3.1.1. Micro syntax

The external representation of languages is frequently described in terms of *identifiers*, *numbers*, *reserved words* and so on. The "micro syntax" of a language defines the syntax of these smallest syntactic entities, subsequently called tokens.

A class of tokens (like identifiers) can in general be described as a regular language and in most textbooks a reference to this fact seems to settle this matter.

But for a formal verification we have to be more specific and devise a precise definitional formalism.

Gries [Gr71] defines a scanner generator. The input to such a program can in some sense be considered to be the formal definition of the micro syntax. However, a more abstract theory is desirable and is developed in chapter III.

To define the scanner formally we define (a) how the input string is to be broken into substrings which represent tokens and (b) how to map each of these strings into a token. Furthermore we define how these two mappings are connected to define the micro syntax.

In the course of this research several ways of giving a comprehensive definition of a scanner have been investigated. It turns out that there is one main problem in most methods: the definition tends to be very large and is very likely to be less comprehensible than a program implementing the scanner. This of course makes the usefulness of the whole verification questionable. For example, the definition of the micro syntax in terms of a finite automaton by specifying a transition table is extremely incomprehensible and error prone.

We adopt the following definitional schema to describe the micro syntax. We define a finite number of regular languages L_i, each of which characterizes a set of tokens. For example the set of identifiers, the set of numbers, the set of delimiters etc. With each language we associate a semantic function S_i which maps elements of L_i into the corresponding token. The name "semantic function" may be confusing in this context; it is to be understood in the sense that the meaning of a character string is a token. For example, a string of digits denotes a numeral. S_i are defined recursively in the style of a denotational definition. Finally, the scanner is given as a function *scan* which is defined in terms of L_i and S_i. An intuitive operational description of *scan* is

- discard all characters which are not an initial substring of any L_i
- find the longest initial substring l of the input such that $l \in L_i$ for some i
- output the token $S_i(l)$ and repeat this process until the input is exhausted.

3.1.2. Phrase structure

Syntax is the best understood part of a language definition. In almost all cases the definition uses a context free grammar [AU72]. Various simplifying notations have been introduced, none of which alters the definitional power of this formalism. The main deficiency of a grammar is that it merely defines the set of syntactically valid programs: it does not specify any particular output other than the derivation tree.

Several very efficient parsing methods are available. In particlular for this

compiler we use *LR — Parsing*[Kn65]. The key idea is to have a parsing table and a program interpreting this table. In some sense this interpreter defines a semantics for parsing tables and the tables themselves could be considered the formal definition of the syntax.

In chapter II we define the notion of a labeled context free grammar. For any unambiguous labeled context free grammar we define a mapping from token sequences to parse trees; labels of productions become labels in these trees. In chapter III the syntax of *LS* is defined in terms of these concepts.

3.1.3. Tree Transformations.

To provide a mapping of derivation trees into more suitable format we introduce tree transformations, functions mapping trees into trees. With this mapping the desired output format for the parser can be specified conveniently

Tree transformations have been studied extensively in the literature [Kr74, Kr75, Ro71]. In particular DeRemer [De73, De74] studied their application in compiler construction. He views the whole translation process as a sequence of tree transformations. The starting point is a forest of singleton trees, the input characters and the result is the machine code. However, since transformations are purely syntactic they cannot serve as a formal definition of the semantics of the source language. A semantics is only given very indirectly by relating a program in source language to an (by definition) equivalent program in target language.

We define tree transformations using recursive functions on trees. We do not put any restrictions on these functions. It turns out, however, that for our source language the functions are of particularly simple nature and allow efficient implementation.

3.2. Semantics

Here the situation is completely different from that in the syntactic realm. Various different methods have been proposed, none of which appears to be generally accepted as a standard definition of semantics. We briefly outline several known methods for the definition of programming language semantics and motivate our particular choice of denotational semantics as a basis for compiler verification.

3.2.1. Operational semantics

The most straightforward way to specify what a program does is to define an interpreter executing the program. Methods following this line are classified as operational definitions [Mc62, Mc63, Re72]. A special case of operational semantics is the Vienna definition method [LW69, We72].

A principal problem with operational semantics is that it does not define the meaning of programs abstractly. Rather, the "meaning of a program can only be determined for a particular set of input data by executing the program with the interpreter. It is not possible to reason about termination of programs on the basis of an operational semantics: if a program loops on a given set of input data then so will the interpreter. But maybe the most serious disadvantage is that an interpreter precludes many implementations of the language. Implementations that do not execute the program in the same way the interpreter does are very hard to verify since it amounts to proving the strong equivalence of two programs, the compiler and the interpreter.

On the other hand operational semantics has several important advantages. It provides a "complete" definition and it is rather easy to comprehend.

3.2.2. Denotational semantics

In contrast mathematical or denotational semantics [Sc70, SS71, Te76, Go79] introduces the meaning of a program as an object, a mapping from input data to results. Thus it is more abstract and allows reasoning about programs in a much more general way, e.g. programs can be compared for equivalence.

We argue that denotational semantics is best suited as a basis for a compiler proof. Firstly, it is the most abstract way of defining a language. As an important consequence a denotational definition relates very easily with other definition methods; other methods are subsumed and equivalence can easily be shown. Secondly, Scott provided a theory for the sound description of higher order functions and self application. Finally denotational definitions have been successfully applied in the definition of several realistic programming languages; amoung these are Pascal [Te77a], Algol60 [Mo75a], Algol68 [Mi72c], SAL [MS76].

The semantics of our source language is defined analogously to that of Pascal in [Te77a]. In particular we use Tennent's separation of the semantics in static and dynamic semantics. However, there are some substantial differences in the way memory management is defined. Our definition is much more suitable as basis for a compiler proof.

3.2.3. Floyd - Hoare logic

In 1969 Hoare introduced an axiomatic system [Ho69] to reason about properties of programs. Since then various extensions to the method have been defined and theoretical results have been established. Of main interest are questions of soundness, completeness and the relation of an axiomatic definition to other methods [HL74, Do76, Co77, Cl79, GM79]. All these questions can only be answered with

respect to a complementary definition of the language.

Although *axiomatics* is very well suited to reason about programs, only in exceptional cases can it serve as a formal definition of a language [Cl79, MG79]. In fact, the method gains much of its strength and usefulness because many subtle issues such as typechecking are purposely left out of consideration.

Even though it is conceivable to construct an axiomatic system which gives a complete detailed definition of all aspects of a language this seems not at all desirable. Attempts to extend axiomatics to completely define complex languages (of the order of Algol68) results in unnatural definitions [Sc78]; a semantics is only provided in a very indirect way.

It is unclear, how a compiler proof based on an axiomatic language definition can be given. A full second order logic seems necessary to prove statements of the form "all input output predicates which are satisfied by the source program are also satisfied by the target program".

3.2.4. Algebraic semantics

Algebraic semantics is based on the observation that the abstract syntax of a programming language is an initial algebra in a class of algebras C. Gouguen et al [GT77]define any other algebra in C to be a possible semantics of the language. The relation between (abstract) syntax and semantics is given by the unique homomorphism between the initial algebra of C and any other member of C. The importance of the algebraic approach is that it unifies most other approaches to semantics. The main problem, however, is the construction of suitable semantics algebras. Denotational semantics is one way of specifying such an algebra.

Most of our presentation could be phrased in algebraic terms. However, we consider this unnecessary for our purpose and prefer not to do so.

An algebraic approach is also useful in the verification of compilers. In this case Cohn's central theorem [Co65, BL70] states that in a suitable algebraic framework (speaking in terms of syntax trees) it is only to prove that terminal nodes are translated correctly to ensure the correctness of the complete translator. This idea has been previously used to prove compiling algorithms (see section on previous work).

For our compiler proof the algebraic approach is not well suited. It requires to talk about the concept of homomorphism which is extremely hard, if not impossible to formalize suitably for present automated verification systems. Instead we use conventional computation induction which in some sense is equivalent (though it is a less abstract notion).

Further details go beyond the scope of this thesis, the reader should consult

[Co65, Go78, GT77, MW72, Mo72, Mo73, Sc76a, TW79, WM72].

3.2.5. Others

Some other methods for formalizing the semantics of programming languages have been proposed which cannot readily be classified in one of the above categories. Two relatively important ones are two-level or van Wijngaarden grammars [Wi69, Wi76] and attribute grammars [Kn68] both concepts are briefly described below.

The basic idea of vanWijngaarden grammars was to extend context free grammars to describe all context dependencies. However aspects of the dynamic semantics are still described informally in English. The main problems in using two-level grammars is that they are fairly difficult to relate to other formalisms. E.g. the definition of mode equality in [Wi76 (section 7.3.1)] and an equivalent first order definition of mode equality are very hard to relate and an equivalence proof is extremely tedious. A further problem is, that for two-level grammars there is no straightforward way of constructing a parser. Most existing Algol68 implementations use a standard context free grammar for their parser and ad hoc procedures to enforce context dependencies.

Another formalism is attribute grammars, first introduced by Knuth in [Kn68]. They have the distinct advantage that it is at least in principle possible to automatically generate an efficient compiler from a language definition in terms of attribute grammars. Several researchers have investigated this possibility and compiler generators are being constructed [Ka76a, Ka76b, Bo76, CH79, De78]. The prime reason for not using this concept in this thesis is that it constitutes a description on a rather low level of abstraction. Also, an attribute grammar does not define a language; rather it relates it to another language (the machine code) by directly describing the translation process. The connection between a set of attributes and an intuitive understanding of a language is very remote. In recent research Bjørner [Bj77] attempts to automatically generate an attribute grammar from a denotational definition. This approach may in the future lead to efficient automatically constructed compilers; at this time, however, it is not useful for the research presented here.

3.3. Machine languages

In principle the techniques used for the source language definition are applicable to the target language as well. However, for the purpose of writing a compiler we are not so very much interested in "parsing" a given program in machine language. Rather, we perform the opposite step. But this is trivial and we omit the definition of a particular external representation for the target language. Consequently, all

we are interested in is the semantics of the target language. As the reader may expect, we use denotational semantics for this purpose.

3.4. Summary

Altogether we define the source language by specifying a function *smean* mapping character strings into their meaning. The meaning of a program is simply a mapping from (input) files to (output) files. If our language had a more complicated file structure a different notion of meaning would be required.

The function *smean* in turn is defined as composition of functions defining the micro syntax, syntax, tree transformations, static, and dynamic semantics respectively.

The micro syntax is defined in terms of a function *scan*, mapping a string of characters into a string of tokens. The syntax is given by the funtion *parse* mapping token strings into a syntax tree. The function *treetr* maps syntax trees into abstract syntax trees. The static semantics is a function *ssem* defined on abstract syntax trees. It is the identity function if the program in question is semantically valid, otherwise *ssem* returns *error*. Finally, *dsem* maps an abstract syntax tree into its meaning, i.e. a function from files to files. Thus the function *smean* is given by

$$smean = dsem \circ ssem \circ treetr \circ parse \circ scan.$$

Clearly, this is an oversimplification. In chapter III the individual functions will be defined more precisely and we will then take care of possible runtime and compile time errors.

The target language is defined similarly by a function *tmean* mapping abstract code sequences into their meaning, again a mapping from input to output files.

4. Developing a verified compiler

Informally the correctness of our compiler can be specified as follows:

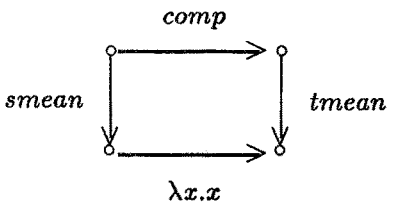
"Whenever the compiler terminates without printing an error message it has generated correct code."

In this section we show how this statement can be formalized, given a formal definition of source and target language.

The above statement is all we prove about the compiler; we do not prove that it ever terminates, that it prints reasonable error messages, etc. Still, the above property is extremely important; it guarantees that the user of the compiler is never deceived into believing his program is translated correctly if it is not.

4.1. What we prove

Source and target language are defined by the functions *smean* and *tmean* mapping a character string and a code sequence into their meaning. The compiler can be looked at as a function *comp* mapping a source program (a string of characters) into a target program (a code sequence). The compiler produces correct code if the following diagram commutes:



where $\lambda x.x$ is the identity function. In other words, the compiler is correct, if the meaning of the source program is equal to the meaning of the produced code. Although the above diagram is helpful in visualizing the correctness of a compiler it is not quite accurate in our case. The compiler to be written need not terminate for all input programs. Also, the compiler may issue meaningless error messages even if the input program is a valid *LS* program. If we consider printing of an error message as non–termination then the correct statement is "given that our compiler terminates then the above diagram commutes".

As described earlier, *smean* is given as a composition

$$smean = dsem \circ ssem \circ treetr \circ parse \circ scan.$$

According to this definition the compiler is divided into four (*treetr* and *parse* are computed in one program module) separate programs executed sequentially. Each program has an input domain and an output domain, such that output and input of two subsequent programs match.

The individual program modules can be thought of as procedures with the following specifications:

procedure *sc*(*in* : *char sequence*; **var** *out* : *token sequence*);
exit *out* = *scan*(*in*)

procedure *ps*(*in* : *token sequence*; **var** *out* : *abstract syntaxtree*);
exit *out* = *treetr*(*parse*(*in*))

procedure *ss*(*in* : *abstract syntactree*; **var** *out* : *abstractsyntaxtree*);
exit *out* = *ssem*(*in*) .

procedure *cg*(*in* : *abstract syntaxtree*; **var** *out* : *code sequence*);
exit *tmean*(*out*) = *dsem*(*in*)

The compiler consists of the sequential execution of these four programs. This can be written as

$$sc(in, ts);\ pa(ts, as_1);\ ss(as_1, as_2);\ cg(as_2, out).$$

After executing this sequence it is true that

$$tsem(out) = dsem(ssem(treetr(parse(scan(in))))) \qquad (*)$$

By the standard interpretation for Hoare formulas this means that if the compiler terminates condition (*) holds. But (*) is just another way of stating that the diagram given above commutes.

A precise specification of the compiler and its components will be given in chapter III.

In the next subsection some principal techniques used in the verification of the compiler are discussed. Following this, we describe the concepts used in the implementation and proof of the individual components of the compiler.

4.2. Representation

Objects manipulated by a computer program are bits. If we are talking about integers, booleans, characters, or even sets or queues we actually are talking about a representation of these objects in terms of bits. For example, an integer number may be represented by a sequence of 32 bits; the value of such a sequence is derived by interpreting the sequence as a binary representation. Alternatively, if we want to talk about arbitrarily large integers, we may have a linked list of elements of a base 1000 representation.

In general an abstract object is represented by specifying a data structure that is used to store such an object and by giving a mapping from this data structure to the desired object. To represent sets for example, we may choose linked list as

representation and define a "representation function" as

$$setrep(l) = \textbf{if } null(l) \textbf{ then } \{\} \textbf{ else } \{car(l)\} \bigcup setrep(cdr(l)).$$

In chapter II we expand on this idea. In particular we are interested how function domains and functions on functions can be represented. In many cases the representation theory can be used to automatically select suitable data structures. A possible application are compiler generators, where data structures could be derived from domain definitions of the denotational semantics.

The idea of representation is relevant for all implementations [Jo79] and proofs. It significantly simplifies the proof of a refinement step, given a proof for the original program. The following trivial example explains this situation. Suppose we are given an abstract procedure *add* with the entry exit specification

$$\{A = A_0\} add(A, b) \{A = A_0 \bigcup \{b\}\}.$$

Given the simple theorem $(A \bigcup \{b\}) \bigcup \{c\} = A \bigcup \{b, c\}$ we can prove the correctness of the program

$$\{A = A_0\} add(A, b); add(A, c) \{A = A_0 \bigcup \{b, c\}\}.$$

Suppose now, that we implement sets as linked lists as above, then *add* operates on a list l instead of a set A. However, the documentation is still written in terms of sets. I.e. the refined version of *add* can be specified as

$$\{setrep(l) = A_0\} add(l, b) \{setrep(l) = A_0 \bigcup \{b\}\}.$$

Using the refined version of *add* does not change the structure of the proof of the using program. The only change is, that the documentation in the using program has to be changed to account for representation functions. The above example becomes

$$\{setrep(l) = A_0\} add(l, b); add(l, c) \{setrep(l) = A_0 \bigcup \{b, c\}\}.$$

No additional theorems are necessary to prove this program. Only in proving the implementation of *add* do we need properties of *setrep*.

4.3. Scanner

For our particular scanner we prove a theorem stating that for any two languages L_i and L_j their respective sets of initial substrings are disjoint. This is useful for an efficient implementation. For example seeing a letter in the input string means that this must be the beginning of an element of the set of identifiers and cannot belong to any other L_i.

In addition we need several trivial theorems about sequences of characters and tokens. They are of the nature of

$$\langle\rangle \cdot x = x \cdot \langle\rangle = x$$
$$x \cdot y = x \cdot z \rightarrow y = z$$

and deserve no further attention.

The implementation follows the recursive definition of *scan* and S_i and uses standard recursion removal techniques. In addition the above theorem about L_i allows several shortcuts to improve efficiency.

4.4. Parser

We use a LR parser [AJ74] to construct the required parse tree. We are not concerned with LR table construction; rather we assume that we have a correct table generator at our disposal.

The theory of LR parsing is fairly well understood; but a formal machine checked proof of an LR parser has not yet been published. The main problem we were faced with was to precisely define the properties of parsing tables required to prove the parser correct. More details are presented in chapter IV.

Again, of course, the parser heavily uses the representation theory and sequences as well as trees.

We talked so much about a stepwise development of program so the reader may be interested in some statistics. We have written and proven 10 successive versions of the parser. The starting point was a barebone version which assumed the correctness of parsing actions and did not produce output. The implementation of parsing actions followed. Next, in several versions we introduced the construction of parse trees and finally added the tree transformation step to produce abstract syntax trees.

The parser is a good example to illustrate the use of virtual code. It would of course be possible to have the parser construct a parse tree and after the program is parsed to apply tree transformations to produce the ultimate output. Instead, the parser produces a parse tree and an abstract syntax tree in parallel; part of the proof is to show that the abstract syntax is the image of the parse tree under the tree transformations at any intermediate stage. We then can prove that the parser satisfies the following specifications

$$\{input = input_0\} pa \{pt = parse(input_0) \wedge ast = treetr(pt)\}.$$

It turns out, that the abstract syntax tree (*ast*) at no point depends on the parse tree (*pt*); rather it is derived directly from the parsing actions performed.

Thus, the parse tree can be virtual code; it never has to be actually computed. The exit specification of the parser then implies that $ast = parse \circ treetr(input_0)$.

4.5. Semantic analysis

The static semantics is defined by recursive functions in Scott's logic of computable functions. In principle the implementation is straightforward but there are two theoretical problems.

The first problem is to express the definition in the assertion language. This is complicated by the fact that we have to deal with higher order functions (i.e. function mapping functions into functions) and that we have to find a consistent treatment of undefined elements($\perp$).

The second problem is what to do with the least fixed point operator (fix) that is used in the definition. If fix is used to define a recursive function then the solution is, of course, to compute this function recursively. The interesting case is, however, when fix is used to describe a data object which has to be computed.

The first problem can be solved fairly easily using the concept of representation functions; details are presented in chapter II.

The key idea to solve the second problem is referred to as "operationalization" of fixed points. Let f be a function from D into D, we write $f \in D \to D$. The least fixed point of this functions $fix\ f$ is an element of D.[6] Given a representation for elements in D and a representation of $f \in D \to D$ the problem is to compute the representation of $fix\ f$. Note, if we are allowed to choose a suitable representation, this problem has always a solution. Just take the symbolic representation f to represent the function f, then $fix\ f$ is a symbolic representation of the fixed point. Of course, we are interested in a solution to this problem for more efficient representations.

Let us look at a simple example. Suppose the domain D is the set of all finite and infinite lists. As a representation we choose Lisp lists and define

$$rep(l) = \textbf{if } null(l) \textbf{ then } \langle\rangle \textbf{ else } \langle car(l)\rangle \cdot rep(cdr(l)).$$

Under rep a circular Lisp list represents an infinite abstract list. Clearly, not all infinite lists can be represented in this way; but a representation is not required to be surjective.

Let us now consider how we can compute the fixed point $fix(\lambda\ x.\langle a\rangle \cdot x)$ which is a list consisting of infinitely many a's. First, we need a representation for functions from lists to lists. We take a pair of two pointers (S-expressions) for this purpose

6.) Precise definition of these notations can be found in chapter II.

and define

$$funcrep(p, q) = \lambda\, l.\ \textbf{if}\ null(p)\ \textbf{then}\ \langle\rangle\ \textbf{else}\ \textbf{if}\ eq(p, q)\ \textbf{then}\ l\ \textbf{else}\ \langle car(p)\rangle \cdot funcrep(p, q)(l)$$

To represent $\lambda\, x.\langle a\rangle \cdot x$ we simply take an arbitrary S-expression s other than *nil* and use the pair $\langle cons(a, s), s\rangle$. Given an arbitrary pair $\langle p, q\rangle$, one can compute $\acute{p}$ such that $rep(\acute{p}) = fix(funcrep(p, q))$ as follows. Find a sublist t of p such that $eq(cdr(t), q)$; compute $rplacd(t, p)$; let $\acute{p} = p$. It is easy to prove that after these steps $\acute{p}$ has the desired property.

Applied to our example, we let $p = cons(a, s)$ and construct $rplacd(p, p)$. Clearly, p now points to a cyclic list which under *rep* represents an infinite list of a's.

In chapter II we formalize the above argument and prove a general theorem that then enables us to efficiently compute the least fixed points required for the semantic analysis.

Given the above theory, the implementation of the semantic analysis phase is straightforward.

4.6. Code generation

4.6.1. Necessary theory

We choose a fairly low level definition of the source language. For example, addressing is done relative to base addresses. For new procedure invocations new base addresses are set up. A display mechanism describes how memory of outer scopes is to be accessed. These concepts are familiar to the compiler writer and are well suited as our specification.

For some statements of *LS* we give a modified definition which is better suited for the verification of the code generation. For example, the definition of the while loop is originally given by a least fixed point. In the revised definition the meaning of the while loop is described by a conditional and explicit jumps. We prove manually that the revised definition is equivalent to the original one.

Also, we give a revised definition of the target language. We add the concept of blocks to the target language. All a block does is to hide labels defined inside from the outside. This concept is advantageous in generating code for example for the while loop. The necessary code requires introduction of labels in the target language that are not exsistent in the source language; it is convenient to be able to hide these in a block.

But we do not generate code for a different language. Rather, we prove that if all labels in a target program with blocks are distinct, then we can omit all

blocks without changing the meaning of the program. The code generation produces "virtual" code containing blocks and real code that is identical except that blocks are omitted. The specification of the code generation is given as

$$\{ast = ast_0\}\ cg$$
$$\{tmean(code_1) = dsem(ast_0) \wedge code_2 = flat(code_1) \wedge distinct(code_1)\}.$$

$code_1$ contains blocks and is proven correct. $code_2$ is generated without blocks but is otherwise identical to $code_1$, expressed as $code_2 = flat(code_1)$. Finally, we prove that all labels in $code_1$ are distinct. Thus the above theorem is applicable and we may conclude $tmean(code_2) = dsem(ast_0)$. Observe, that this reasoning is anologous to that in the parser.

4.6.2. Implementation

With the above theory at hand the actual implementation is fairly simple. We have a set of procedures to generate fixed code sequences for primitive operations such as updating a location, accessing the value of a location and so on.

Code for more compound entities is generated recursively. Let us illustrate this with a very simple example. Suppose the recursive procedure *code* generates code for expressions. We have the specification

$$\{true\}code(E, z)\{\text{“}z \text{ computes } E\text{, puts result on stack”}\}.$$

Inside *code* we have a branch which deals with binary operations:

```
if E = "E1 + E2" then
        begin
            code(E1, z1);
            code(E2, z2);
            primcode(+, z3);
            z ← z1·z2·z3;
        end
```

primcode is a procedure generating code for primitive operations; in this case for "+" it produces code that takes the two top elements of the stack and pushes back their sum. Although this example is very much simplified, it demonstrates the basic principle of the code generation and its proof.

The reader may wonder about one more problem; how are fixed points of the semantic definition handled in the code generation. The solution is simple in this case. The meaning of a loop in the source language is described as a fixed point, say $fix\ s$. For this loop we generate code whose meaning in the target language is

defined as a fixed point as well, say $fix\ t$. To show that both fixed points are equal we merely have to prove that $t = s$.

5. Related work

5.1. Previous work on compiler verification

Correct compilers have been of great interest to researchers from the very beginning of verification ideas. We summarize the work that has been done in this area and give a short characterization of the particular innovations introduced in each case.

The first attempt to verify a compiler or rather a translation algorithm, has been undertaken by McCarthy and Painter [MP66] in 1966 (see also [Pa67]). Though limited in scope this work has been of great impact and many researchers have directly built upon it. The language considered are arithmetic expressions. The compiler is a set of recursive functions and the proof proceeds by recursion induction.[7] The underlying semantical definition is operational.

A machine checked proof of the McCarthy/Painter compiler has been done by W. Diffie but has not been published.

Kaplan [Ka67] has built on McCarthy/Painter and proved a compiler for a yet more complex language including assignments and loops, still using the same proof technique. Again in [Bu69] R. Burstall proves the McCarthy/Painter compiler by structural induction. But as he points out, the proof is very similar to one given by recursion induction.

A completely new aspect enters the field of compiler verification in 1969 when Burstall and Landin [BL69] apply an algebraic method to greatly reduce the amount of work required in proving a compiler.

The same proof method is used by Milner and Weyhrauch [MW72, WM72] in 1972 to verify a compiler for an algolic language. But they introduce yet another innovation: semantics and proof are described in Scott's logic of computable functions. F.L. Morris uses the same algebraic method in [Mo72, Mo73]. More recently Thatcher, Wagner, and Wright [TW79] put forth similar ideas.

The proof techniques used by McCarthy and Painter are again employed by London [Lo71, Lo72] to prove an existing compiler for a simple version of LISP.

In recent years some authors tried to use axiomatic semantics as a basis for a compiler proof [Ch76, CM75]. But there appear to be some difficult problems.

7.) different recursion principles are discussed in chapter II.

There is no notion of semantics in these proofs at all. Source and target languages are "defined" by a set of rules. In addition, static semantic issues are totally ignored. Again, the language handled is not much more interesting than that of the McCarthy/Painter compiler.

Lynn [Ly78] is the first to consider the problem of user defined functions in a compiler for LISP. He too uses Hoare style proof rules to specify the semantics of source and target language.

Yet a different approach is taken by Boyer and Moore. Their LISP theorem prover [BM77] has been shown to be capable of verifying an optimizing compiler for LISP expressions. Their system is capable of automatically synthesizing induction hypotheses for suitable recursive functions.

Two researchers continued to use denotational semantics as a basis for a compiler proof. First there are Milne and Strachey which in their book [MS76] give a proof of a compiling algorithm for a language of the complexity of Algol68. The proof is given completely by hand and the target language is a hypothetical machine language. Avra Cohn [Co79a, Co79b] proves the correctness of several components of a compiling algorithm using the Edinburgh implementation of LCF [GM75]. One problem she is interested in is the compilation of recursive procedures into stack implementations. Her main emphasis, though, is on the technique of using LCF and automating proofs in LCF.

A totally different approach is taken by H. Samet [Sa75]. Instead of proving the correctness of a compiler he proves equivalence of a particular source program and the produced code. Although this approach is in general much more expensive than verifying a compiler it is useful in situation where a correct translation is imperative and a correct compiler is not available.

5.2. Relation to our work

Most practitioners would not consider "compilers" verified previously to be "real compilers". Our emphasis is to verify a compiler that is – at least in principle – a practical, usable compiler. Therefore we choose a realistic source language in which a programmer might want to actually program. The only language comparable in size and complexity is *SAL*, considered by Milne and Strachey in [MS76]. But they consider a very abstract compiler only and their attention is limited to code generation, all proves are manual.

Milne and Strachey use a semantic definition of a higher level of abstraction than the definition of *LS* used in this work. If we were to use a more abstract definition a lower level definition would have to be developed as part of the theory for the code generation. The techniques and results in [MS76] are directly applicable for

the necessary equivalence proof. In this sense the work in [MS76] is complementary to ours.

Cohn's work represents an important step towards mechanically checking proofs of the kind we give manually to derive theorems necessary for the code generation.

For our compiler to be practical we consider not only the code generation part but also verify a scanner, parser, and static semantic analysis. These issues have not been dealt with previously.

Although the development of a suitable logical basis for our compiler verification is done manually all actual program proofs are completely machine checked. No comparable mechanical verification attempt is known to the author.

An important part of this research deals with technical issues that are prerequisits for a mechanical compiler verification. These problems have not been considered before in this form. Some main points are

- formalization of compiler correctness suitable as input for a mechanical verification system,
- structuring of programs such that mechanical verification becomes manageable, and
- development of specialized verification techniques to deal with pointer operations, least fixed points, and quantification.

5.3. Compiler generators

It would, of course, be much more efficient could we verify a compiler generator. We then could generate correct compilers for any specification. But, given the current state of the art, this goal is somewhat unrealistic. We do not even know how to write compiler generators rather than how to verify them. Though much has been written about this subject no practical system exists to my knowledge. Most system called compiler generators turn out to be just parser generators.

Important work towards automatic generation of compilers has been done though. We should mention P. Mosses' thesis [Mo75b] and the work based on attribute grammars [Ka76a, Ka76b, Bo76, CH79, Bj77].

Our work is not totally unrelated to compiler generation. Firstly, the parsing algorithm implemented operates table driven; thus it remains correct for any language that can be defined by an LR(1) grammar.

Further, we show how a program and its proof can be systematically derived from a denotational semantics. These results give some indication as to how this process or at least some parts of it can be automated.

6. Organization of this thesis

In chapter II we introduce notations and relevant theories required for the compiler proof. Among other things we consider

- first order logic and its application as assertion language
- Scott's logic of computable functions
- Axiomatization of Scottery in logic
- Proofs in logic and Scottery and their relation
- Theory of representation
- Treatment of fixed points

In chapter III we describe the source and target languages *LS* and *LT*. First, we give an informal introduction to the source language followed by the formal definition of scanner, parser, static and dynamic semantics. For each of these parts we define the definitional formalism used. We introduce a hypothetical machine executing the target language *LT*. A formal definition of *LT* is given.

Chaper IV describes precisely how the correctness of the compiler and its components is specified in a format accepted by a mechanical verifier. We then discuss the systematic development of the individual components and demonstrate the proof principles employed.

In the concluding chapter V we summarize our results and consider possible changes, improvements and extensions of the compiler. We discuss implications of this work on the design of verification systems and more sophisticated program development systems that encompass the whole design cycle of programs.